Q4: CI/CD, OIDC i testy wdrożenia

Jak rozumieć pytanie, rozpoznawać scenariusze i oceniać odpowiedzi

Data Platform Engineer / AWS DevOps · przewodnik po polsku · 17 października 2026

Materiał dla prowadzącego bez praktyki DevOps. Tłumaczy drogę od zmiany kodu do działającego przetwarzania danych, logowanie pipeline'u do AWS przez OIDC oraz różnicę między istnieniem zasobów a użytecznym wynikiem. Pytania do kandydata są po angielsku.

W planie rozmowy Q4 zajmuje 7 minut. To bank wyjaśnień i scenariuszy do przygotowania, a nie lista wszystkich pytań do odczytania. Przykłady nie są potwierdzonym opisem środowiska firmy ani doświadczenia kandydata.

Najważniejsze: ustal, jaka wersja jest wdrażana, kto może ją wdrożyć i jaki test dowodzi, że odbiorca otrzymuje poprawne dane.

Spis treści

1. Pytanie główne: o co właściwie pytasz 2
2. Słownik od podstaw: CI, CD, pipeline i artefakt 2
3. Jeden konkretny przepływ od PR do produkcji 3
4. Co to jest OIDC i jaki problem rozwiązuje 3
5. Trust policy a permissions policy 4
6. Claims, aud, sub i id-token: write bez zagadki ze składni 4
7. Akceptacja produkcji i zaufanie to dwa zabezpieczenia 5
8. Co naprawdę sprawdzają pytest, mocki i integracja 5
9. Zasoby istnieją, ale analityk nie czyta danych 6
10. Nowa wersja zadania: schema i zgodność 6
11. Apply się udał, test integracyjny nie 7
12. Możliwe scenariusze odpowiedzi kandydata 7
13. Jak poprowadzić 7 minut i ocenić poziom 8
14. Ściąga przed zadaniem pytania 9

1. Pytanie główne: o co właściwie pytasz

> “Your CV mentions OIDC and post-deployment pytest checks. Describe the pipeline from a pull request to production. How do you deploy Terraform and a versioned Lambda script or Glue job, what do you test, who approves the change, and how does the CI job obtain access to AWS?”

Po polsku: kandydat deklaruje pracę z GitHub Actions, OIDC i pytest. Ma połączyć te elementy w rzeczywisty proces: zmiana, kontrola, dostęp do AWS, wdrożenie infrastruktury i kodu zadania, a potem sprawdzenie wyniku.

Nie musisz znać składni YAML ani pisać workflow. Sprawdzasz, czy potrafi wyjaśnić co uruchamia proces, która tożsamość wykonuje zmianę i co blokuje złą wersję.

Obszar	Czego szukasz
Przebieg	Powiązanie PR, review, artefaktu, środowiska i zatwierdzenia
Dostęp	Tymczasowe poświadczenia oraz ograniczona możliwość przyjęcia roli
Testy	Konkretne zachowanie, tożsamość odbiorcy i weryfikacja danych
Awaria	Zatrzymanie promocji i świadoma decyzja o naprawie lub cofnięciu

2. Słownik od podstaw: CI, CD, pipeline i artefakt

CI, continuous integration: automatyczne sprawdzanie zmian, np. testy Pythona i kontrola konfiguracji. **CD, continuous delivery/deployment:** przygotowanie i przeprowadzanie wdrożeń. W praktyce projekt może wymagać ręcznej akceptacji produkcji; samo użycie skrótu CD nie rozstrzyga tej kwestii.

Pipeline/workflow to zapisana sekwencja kroków. **Job** jest jednostką wykonania tych kroków. **Runner** to środowisko, na którym komendy działają. Nie utożsamiaj GitHub Actions z uprawnieniami administratora AWS.

Artefakt to przygotowany wynik budowania: np. paczka kodu Lambdy albo wersjonowany skrypt Glue. **Promocja** oznacza dopuszczenie sprawdzonej wersji do kolejnego środowiska. **Approval** to zatwierdzenie przez uprawnioną osobę lub uzgodniony mechanizm kontroli.

Przykład: commit A daje paczkę o identyfikatorze X. Testujemy X w test i wdrażamy X do prod, z produkcyjną konfiguracją. Jeśli ponowne budowanie wytworzyło coś innego, nie wystarczy powiedzieć „to ten sam branch”.

> “How do you identify the exact artifact that reached production?”

Dobra odpowiedź: wersja, commit, digest lub inne jednoznaczne powiązanie. Nazwa `latest` bez historii i możliwości ustalenia zawartości jest słabym dowodem.

3. Jeden konkretny przepływ od PR do produkcji

Załóżmy, że zmiana dodaje kolumnę do przygotowanego zbioru orders i potrzebuje nowego uprawnienia roli zadania. Są dwa rodzaje zmian: kod przetwarzania i infrastruktura. Kandydat powinien kontrolować ich zgodność.

1. PR zawiera zmianę Pythona, konfiguracji Terraform i oczekiwanego schematu. Review ocenia logikę oraz zakres uprawnień.
2. CI uruchamia szybkie testy i kontrole. Wytwarza identyfikowalny artefakt.
3. Plan Terraform pokazuje wpływ na środowisko. Kandydat weryfikuje konto, rolę i operacje, szczególnie wymianę lub usunięcie zasobów.
4. W dev/test wdrażana jest zgodna para infrastruktury i kodu. Mały zestaw danych uruchamia przetwarzanie i sprawdzenie wyniku.
5. Dodatkowe testy używają docelowego typu tożsamości odbiorcy: poprawny odczyt dozwolonego zbioru i odmowa dostępu do niedozwolonego.
6. Po spełnieniu bramek zatwierdzana jest produkcja. Wdrażana jest ustalona wersja; inne konfiguracje środowiska są jawne i kontrolowane.
7. Sprawdzenie po wdrożeniu potwierdza działanie i świeżość wyniku. Błąd uruchamia reakcję, a nie tylko zmianę koloru statusu w interfejsie.

To przykład, nie jedyny poprawny układ. Kandydat może mieć oddzielne repozytoria lub procesy dla IaC i kodu, jeśli potrafi wyjaśnić ich koordynację.

4. Co to jest OIDC i jaki problem rozwiązuje

OIDC, OpenID Connect, pozwala AWS zaufać potwierdzonej tożsamości konkretnego uruchomienia workflow. GitHub wystawia token opisujący kontekst joba. AWS sprawdza go i, jeśli warunki zaufania pasują, STS wydaje tymczasowe poświadczenia roli. Nie trzeba trzymać stałego klucza użytkownika IAM w repozytorium. GitHub: OIDC.

Wyobraź sobie wejście do biura: potwierdzenie, kim jesteś, jest oddzielne od listy pokoi, do których wolno ci wejść. W AWS odpowiadają temu dwa pytania: **kto może przyjąć rolę** oraz **co ta rola może zrobić**.

Przebieg krok po kroku:

1. Wystartował właściwy job, np. wdrożenie do produkcji z dozwolonego kontekstu.
2. Ma pozwolenie GitHub na pobranie tokenu OIDC.
3. Token trafia do AWS STS w żądaniu przyjęcia roli.
4. AWS weryfikuje wystawcę, przeznaczenie tokenu i warunki trust policy.
5. Przy zgodności wydaje poświadczenia z ograniczonym czasem ważności.
6. Komendy korzystają z uprawnień tej roli, np. do wdrożenia określonych zasobów.

Ważne: krótkotrwałe poświadczenie może nadal mieć zbyt szerokie uprawnienia. OIDC ogranicza problem stałych kluczy, ale nie naprawia automatycznie polityk dostępu.

5. Trust policy a permissions policy

Trust policy roli ustala, kto i na jakich warunkach może ją przyjąć. Polityki uprawnień określają operacje dostępne po jej przyjęciu. Ostateczny dostęp zależy też od innych właściwych ograniczeń, np. jawnego deny.

> “How does the role's trust policy differ from the permissions granted to that role?”

Sytuacja	Co sprawdzić
Job nie może uzyskać poświadczeń	Token, dostawca OIDC, warunki zaufania, kontekst uruchomienia
Poświadczenia uzyskał, ale S3 odmawia odczytu	Rzeczywista rola, operacja, zasób i właściwe warstwy uprawnień
Każdy projekt organizacji może przyjąć rolę prod	Zaufanie jest prawdopodobnie zbyt szerokie
Zaufanie jest wąskie, ale rola ma AdministratorAccess	Nadal zbyt duży potencjalny wpływ przejętego wdrożenia

Dobre dopytanie:

> “Could an unrelated repository or an untrusted pull request obtain the same production role?”

Szukasz ograniczenia do właściwego repozytorium i kontekstu wdrożenia, kontroli uruchamianego kodu oraz rozdzielenia ról środowisk. Samo stwierdzenie „repozytorium jest prywatne” nie wyjaśnia mechanizmu zaufania.

6. Claims, aud, sub i id-token: write bez zagadki ze składni

Claims to informacje zawarte w tokenie. `aud` wskazuje odbiorcę, dla którego token jest przeznaczony, a `sub` opisuje jego podmiot/kontekst. Trust policy powinna dopuszczać oczekiwany kontekst zamiast dowolnego uruchomienia GitHub. `id-token: write` pozwala jobowi poprosić o token OIDC; nie daje prawa zapisu do AWS. GitHub: OIDC w AWS.

> “What restricts role assumption to this repository and deployment context?”

Akceptuj wyjaśnienie własnymi słowami, np. „sprawdzamy claims repozytorium i środowiska w warunkach zaufania”. Dokładna postać `sub` zależy od konfiguracji; nie wymagaj zapamiętanego ciągu znaków ani szczegółowego JSON-a.

Słaba odpowiedź: „id-token write pozwala zapisywać do S3”. Po dopytaniu kandydat powinien odróżnić możliwość pobrania tokenu od uprawnień przyjętej roli.

Dobra odpowiedź z granicą doświadczenia: „Nie konfigurowałem providera, ale zmieniałem workflow i wiem, że job przyjmuje tę rolę tylko z tego kontekstu”. Sprawdź, czy potrafi pokazać mechanizm i swoją część pracy, zamiast karać za brak administracji całym kontem.

7. Akceptacja produkcji i zaufanie to dwa zabezpieczenia

Zatwierdzenie środowiska GitHub może zatrzymać job do czasu review. Dodatkowe reguły określają dopuszczalne gałęzie lub tagi; dostępność funkcji zależy od konfiguracji i planu GitHub. GitHub: środowiska wdrożeń.

Nie wystarczy jednak powiedzieć „mamy approval”, jeśli inne uruchomienie może ominąć tę ścieżkę i uzyskać tę samą rolę AWS. Z drugiej strony sama poprawna trust policy nie dowodzi, że ktoś ocenił zakres zmiany infrastruktury.

> “How are the environment approval and the AWS role conditions connected?”

Dobry kandydat opisze oba mechanizmy i ich konfigurację. Jeżeli token jest powiązany z environment, nie zakładaj, że jego `sub` jednocześnie dowodzi konkretnej gałęzi; ograniczenia gałęzi trzeba spójnie zapewnić w procesie.

Zapytaj też, czy po zatwierdzeniu zmienił się kod lub plan. Zgoda na jedną wersję nie powinna być przepustką do wdrożenia dowolnej późniejszej zawartości.

8. Co naprawdę sprawdzają pytest, mocki i integracja

pytest to narzędzie uruchamiające testy. Nazwa nie mówi, czy test sprawdza funkcję, AWS czy dane. Można napisać zarówno bardzo użyteczny test, jak i test potwierdzający wyłącznie istnienie bucketa.

Rodzaj sprawdzenia	Przykład	Czego nie dowodzi
Jednostkowe	Funkcja dobrze wybiera błędne rekordy	Że rola ma dostęp do S3
Z mockiem	Symulowany timeout prowadzi do zgłoszenia błędu	Że prawdziwy endpoint jest osiągalny
Integracyjne w AWS	Zapis wejścia uruchamia zadanie i daje oczekiwany wynik	Pełnej poprawności dla wszystkich danych produkcji
Odczyt przez konsumenta	Rola analityka czyta dozwolony wynik	Że nie ma dostępu do innego zbioru, jeśli tego nie sprawdzono

Mock zastępuje zależność kontrolowaną odpowiedzią. Dzięki temu łatwiej sprawdzić trudny błąd, ale testuje się również założenia tej symulacji. pytest: monkeypatch.

> “What does this specific test prove, and what would still require a real AWS check?”

Dobra odpowiedź wskazuje granicę dowodu. Nie wymagaj konkretnej biblioteki mocków.

9. Zasoby istnieją, ale analityk nie czyta danych

Bucket jest, zadanie ma status sukcesu, test wykonany rolą wdrożeniową odczytał wynik. Analityk nadal dostaje AccessDenied. Czy test był dobry?

Potwierdził część działania, ale używał innej tożsamości. Rola wdrożeniowa może mieć uprawnienia, których konsument nie ma. Potrzebne jest sprawdzenie reprezentujące właściwego odbiorcę, jego konto, zasób i szyfrowanie.

> “What test would detect that resources exist but the intended consumer still cannot read the data?”

Przykładowy scenariusz testowy:

1. Powstaje niewrażliwy plik testowy i jego oczekiwany wynik w wydzielonym zakresie.
2. Test używa kontrolowanej tożsamości odpowiadającej konsumentowi.
3. Czyta dozwolony zbiór i sprawdza oczekiwaną zawartość lub schemat.
4. Próbuje uzgodnionej operacji poza zakresem i oczekuje odmowy.
5. Usuwa tylko własne dane testowe albo korzysta z uzgodnionej retencji.

Nie trzeba używać prawdziwego konta konkretnego pracownika. Trzeba odwzorować istotny model uprawnień. Test nie powinien nadawać sobie administratora, żeby przypadkiem „zrobić zielono”.

10. Nowa wersja zadania: schema i zgodność

> “How would you test a new job version against representative data and the expected schema before promotion?”

Schema to umowa o kształcie danych: nazwy kolumn, typy, pola wymagane, ewentualnie znaczenie i klucze. Przykład: `amount` zmienia się z liczby na tekst. Zadanie może zapisać plik bez błędu, a raport przestaje poprawnie sumować.

Dobre testy obejmują poprawny przykład, puste wejście, brak wymaganej kolumny, nieprawidłowy typ, duplikat i przypadek zmiany wersji. Reprezentatywne dane oznaczają istotne przypadki, a nie przypadkowy wielki plik produkcyjny.

Sprawdzenie powinno porównać wynik z oczekiwaniem: schemat, reguły jakości, liczbę poprawnie obsłużonych rekordów lub konkretne wartości. W razie zmiany kontraktu trzeba uwzględnić odbiorców i kolejność wdrożeń.

> “Can the previous consumer read the new output, and what happens during a mixed-version deployment?”

To dopytanie dodatkowe. Na poziomie regular/mid wystarczy rozpoznanie problemu zgodności i sensowny sposób sprawdzenia. Nie wymagaj projektowania kompletnego systemu wersjonowania wszystkich schematów w siedem minut.

11. Apply się udał, test integracyjny nie

> “What would you do if apply succeeded but the integration test failed?”

Dobra odpowiedź zaczyna się od zatrzymania promocji i zebrania dowodów. Potem trzeba ustalić, czy błąd dotyczy aplikacji, konfiguracji, dostępu, danych, samego testu czy oczekiwania na asynchroniczny wynik.

Przyczyna	Przykładowa reakcja
Brak uprawnienia odbiorcy	Poprawić właściwą politykę i ponowić sprawdzenie
Błędny skrypt	Cofnąć do zgodnej wersji lub wdrożyć małą naprawę
Test odczytuje wynik zbyt wcześnie	Poprawić ograniczone czasowo oczekiwanie, ustalić realny budżet czasu
Nowa wersja zapisała błędne dane	Zatrzymać dalszy wpływ, zidentyfikować zakres, naprawić dane i przetwarzanie

Rollback kodu nie usuwa automatycznie niepoprawnych wyników już zapisanych w danych. Cofnięcie infrastruktury może też usunąć potrzebny zasób. Decyzja o fix forward, czyli poprawce do przodu, powinna wynikać ze skutków i zgodności.

> “What has already changed in the data, and which part of that would your rollback restore?”

Jeśli kandydat proponuje ignorowanie testu, zapytaj o dowód jego błędności i kontrolowany proces wyjątku. Sama presja terminu nie potwierdza sprawności systemu.

12. Możliwe scenariusze odpowiedzi kandydata

Wypowiedź	Dopytanie po angielsku	Na co zwrócić uwagę
“I used an existing workflow.”	“Which step did you change and how did you verify it?”	Własny wkład może być wąski, ale konkretny
“We used OIDC.”	“Walk me from the job identity to the AWS API call.”	Mechanizm, nie samo hasło
“The secret contained AWS keys.”	“Why were long-lived keys needed, and how were they restricted and rotated?”	Rzeczywisty wyjątek lub brak migracji, bez automatycznej dyskwalifikacji
“pytest passed.”	“Which assertion would fail if the output schema were wrong?”	Co sprawdzał test
“Everything was mocked.”	“What real environment checks covered IAM, KMS and connectivity?”	Rozumienie ograniczeń
“The platform team configured access.”	“What could you diagnose yourself when role assumption failed?”	Granica samodzielności
“I deploy the latest script.”	“How would you identify and restore the previously tested version?”	Kontrola wersji i odtwarzalność

Przykładowa dobra odpowiedź:

> “The PR runs checks and builds a versioned job artifact. We review the Terraform plan and deploy to a test environment. The job obtains temporary AWS credentials through OIDC and a role restricted to the expected repository and deployment context. We test representative inputs and read the result using the intended consumer permissions. Production requires approval and uses the identified artifact. If a post-deployment test fails, we stop promotion, investigate and choose a safe correction based on what changed.”

Nie wymagaj tych samych słów. Dopytaj o jeden faktyczny błąd z doświadczenia. Różnica między „zbudowałem” i „korzystałem” wpływa na ocenę samodzielności, ale oba doświadczenia mogą być przydatne.

13. Jak poprowadzić 7 minut i ocenić poziom

Czas	Zadanie
0-2 min	Pipeline, artefakt, infrastruktura i promocja
2-4 min	OIDC: skąd poświadczenia i co ogranicza rolę
4-6 min	Jeden konkretny test i przypadek apply-success/test-failure
6-7 min	Najważniejsze wyjaśnienie oraz zapis dowodu

Poziom 3 / wystarczająca samodzielność w typowym przypadku: wyjaśnia spójny pipeline, tymczasowy dostęp z ograniczonym zaufaniem, konkretny test wyniku i zatrzymanie błędnej promocji. Nie musi pamiętać dokładnego `sub`.

Poziom 2: zna proces, ale potrzebuje podpowiedzi do rozróżnienia zaufania i uprawnień albo do wskazania, co właściwie testuje pytest.

Poziom 4: uzasadnia dobór bramek, zgodność wersji i reakcję na częściowy sukces, podając konkretny przykład. **Poziom 1:** po wyjaśnieniu nadal zakłada, że dowolny token daje AWS admin lub zielony apply dowodzi poprawnych danych. **N/O:** nie zebrano dowodu; nie zastępuj tego zerem.

Zapisz: wersja/artefakt, tożsamość, warunek zaufania, konkretna asercja testu, reakcja na awarię i zakres podpowiedzi. Nie oceniaj samej liczby nazw usług.

14. Ściąga przed zadaniem pytania

Po co pytasz: sprawdzić, czy kandydat rozumie bezpieczne dostarczenie sprawdzonej wersji przetwarzania danych do AWS.

Słownik: CI = automatyczne kontrole; pipeline = kroki procesu; artefakt = wersjonowany wynik budowania; OIDC = potwierdzenie tożsamości joba; STS = usługa wydająca tymczasowe poświadczenia; trust policy = kto może przyjąć rolę; permissions = co potem może zrobić; mock = zastępstwo zależności w teście.

Minimum dobrej odpowiedzi:

- Wiadomo, jaki kod, artefakt i konfiguracja trafiają do produkcji.
- Job otrzymuje tymczasowe poświadczenia z ograniczonego kontekstu.
- `id-token: write` nie daje samodzielnie uprawnień AWS.
- Test sprawdza konkretny wynik i właściwą tożsamość odbiorcy.
- Błąd po apply zatrzymuje promocję; naprawa uwzględnia zapisane dane.

Dwa najlepsze dopytania:

> “What prevents an unrelated workflow from assuming the production role?”

> “Which test would fail if the intended analyst could not read the new dataset?”

Sygnały do wyjaśnienia: administrator jako standardowa rola, token bez kontroli kontekstu, „pytest” bez żadnej asercji, nieidentyfikowalny `latest`, automatyczny rollback bez rozważenia danych.

Nie wymagaj: pełnego YAML, JSON trust policy ani nazw wszystkich narzędzi. Liczy się mechanizm, własny wkład i dowód działania.

Materiał powiązany: plan rozmowy.